

Jaiden Camp

Jason Stys

The inputs are numbers from 0 - 9 each representing something different

0 -> airplane

1 -> automobile

2 -> bird

3 -> cat

4 -> deer

5 -> dog

6 -> frog

7 -> horse

8 -> ship

9 -> truck

The dataset is 50,000 images that has a 32x32 dimensions

```
import numpy as np
import matplotlib.pyplot as plt
import keras
from keras.datasets import cifar10
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.layers import BatchNormalization
from keras.optimizers import SGD, Ftrl
from matplotlib import pyplot
import pandas as pd
```

```
(x_train, y_train), (x_valid, y_valid) = cifar10.load_data()

x_train = x_train.reshape(50000, 32, 32, 3).astype('float32')
x_valid = x_valid.reshape(10000, 32, 32, 3).astype('float32')

x_train /= 255
x_valid /= 255
n_classes = 10

# y_train and y_valid are left as integer labels for sparse_categorical_crossentropy
y_train = keras.utils.to_categorical(y_train, n_classes)
y_valid = keras.utils.to_categorical(y_valid, n_classes)
```

relu = 43.01

softplus = 48.32

sigmoid = 48.22

```
model = Sequential()
model.add(Flatten(input_shape = (32, 32, 3)))
model.add(Dense(128, activation = 'softplus'))
model.add(Dense(128, activation = 'softplus'))
model.add(Dense(10, activation = 'softmax'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to the Flatten layer.
super().__init__(**kwargs)
```

128 Neurons = 51.37... w/ 2 layers = 56.37

256 Neurons = 50.55... w/ 2 layers = 53.09

512 Neurons = 51.48... w/ 2 layers = 51.53

1024 Neurons = 50.67

```
model.compile(loss = 'categorical_crossentropy', optimizer=SGD(learning_rate=0.1), metrics=['accuracy'])
```

```
model.fit(x_train, y_train, batch_size=128, epochs=20, verbose=1, validation_data=(x_valid, y_valid))
```

```

Epoch 1/20
391/391 ————— 6s 14ms/step - accuracy: 0.2212 - loss: 2.1557 - val_accuracy: 0.3055 - val_loss: 1.8693
Epoch 2/20
391/391 ————— 6s 16ms/step - accuracy: 0.3386 - loss: 1.8377 - val_accuracy: 0.3499 - val_loss: 1.7685
Epoch 3/20
391/391 ————— 5s 12ms/step - accuracy: 0.3736 - loss: 1.7466 - val_accuracy: 0.3772 - val_loss: 1.7209
Epoch 4/20
391/391 ————— 6s 15ms/step - accuracy: 0.3983 - loss: 1.6824 - val_accuracy: 0.3971 - val_loss: 1.6760
Epoch 5/20
391/391 ————— 5s 12ms/step - accuracy: 0.4177 - loss: 1.6318 - val_accuracy: 0.4122 - val_loss: 1.6369
Epoch 6/20
391/391 ————— 6s 16ms/step - accuracy: 0.4324 - loss: 1.5897 - val_accuracy: 0.4261 - val_loss: 1.6049
Epoch 7/20
391/391 ————— 5s 12ms/step - accuracy: 0.4449 - loss: 1.5559 - val_accuracy: 0.4339 - val_loss: 1.5791
Epoch 8/20
391/391 ————— 5s 12ms/step - accuracy: 0.4565 - loss: 1.5284 - val_accuracy: 0.4407 - val_loss: 1.5579
Epoch 9/20
391/391 ————— 6s 14ms/step - accuracy: 0.4631 - loss: 1.5050 - val_accuracy: 0.4467 - val_loss: 1.5403
Epoch 10/20
391/391 ————— 5s 13ms/step - accuracy: 0.4696 - loss: 1.4841 - val_accuracy: 0.4522 - val_loss: 1.5257
Epoch 11/20
391/391 ————— 6s 16ms/step - accuracy: 0.4764 - loss: 1.4648 - val_accuracy: 0.4573 - val_loss: 1.5136
Epoch 12/20
391/391 ————— 8s 12ms/step - accuracy: 0.4827 - loss: 1.4467 - val_accuracy: 0.4606 - val_loss: 1.5036
Epoch 13/20
391/391 ————— 6s 14ms/step - accuracy: 0.4887 - loss: 1.4295 - val_accuracy: 0.4652 - val_loss: 1.4951
Epoch 14/20
391/391 ————— 4s 11ms/step - accuracy: 0.4943 - loss: 1.4131 - val_accuracy: 0.4665 - val_loss: 1.4877
Epoch 15/20
391/391 ————— 5s 14ms/step - accuracy: 0.4997 - loss: 1.3974 - val_accuracy: 0.4686 - val_loss: 1.4809
Epoch 16/20
391/391 ————— 5s 12ms/step - accuracy: 0.5053 - loss: 1.3824 - val_accuracy: 0.4727 - val_loss: 1.4744
Epoch 17/20
391/391 ————— 5s 12ms/step - accuracy: 0.5104 - loss: 1.3678 - val_accuracy: 0.4769 - val_loss: 1.4682
Epoch 18/20
391/391 ————— 6s 14ms/step - accuracy: 0.5156 - loss: 1.3538 - val_accuracy: 0.4789 - val_loss: 1.4623
Epoch 19/20
391/391 ————— 4s 11ms/step - accuracy: 0.5209 - loss: 1.3401 - val_accuracy: 0.4818 - val_loss: 1.4568
Epoch 20/20
391/391 ————— 6s 13ms/step - accuracy: 0.5261 - loss: 1.3267 - val_accuracy: 0.4842 - val_loss: 1.4517
<keras.src.callbacks.history.History at 0x78dc3c5873e0>

```

The first way I tried to optimize was changing the amount of neurons in each layer (ranging from 64 - 1024). The ideal level was 256. Next I tried a ranging level of neurons with 2 levels. The ideal number was 128. Next I tried using different loss functions with categorical_entropy being the most accurate. Finally, I tried a list of different optimizers and the SGD optimizer was the most accurate.

```

#CIFAR-10 load + inspection + preprocessing

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import keras

from keras.datasets import cifar10
from keras.utils import to_categorical

# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# Load CIFAR-10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

print("x_train:", x_train.shape, x_train.dtype)
print("y_train:", y_train.shape, y_train.dtype)
print("x_test: ", x_test.shape, x_test.dtype)
print("y_test: ", y_test.shape, y_test.dtype)

# CIFAR-10 label names (0-9)
class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]

# Show a few samples
plt.figure(figsize=(7, 7))
for i in range(9):
    plt.subplot(3, 3, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[int(y_train[i])])
    plt.axis("off")
plt.tight_layout()
plt.show()

```

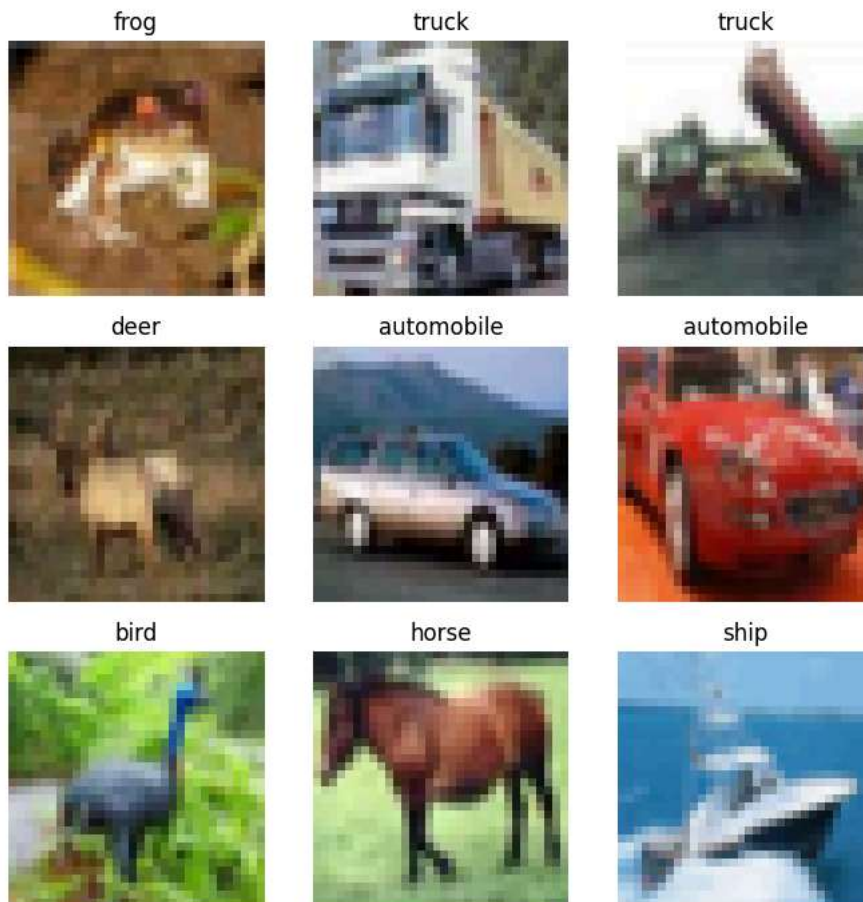
```
# Preprocess: float + scale to [0, 1]
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# One-hot encode labels
y_train = y_train.reshape(-1) # (50000,)
y_test = y_test.reshape(-1) # (10000,)

n_classes = 10
y_train_cat = to_categorical(y_train, n_classes) # (50000, 10)
y_test_cat = to_categorical(y_test, n_classes) # (10000, 10)

print("After preprocessing:")
print("x_train:", x_train.shape, x_train.dtype, "min/max:", x_train.min(), x_train.max())
print("y_train_cat:", y_train_cat.shape, y_train_cat.dtype)
```

```
x_train: (50000, 32, 32, 3) uint8
y_train: (50000, 1) uint8
x_test: (10000, 32, 32, 3) uint8
y_test: (10000, 1) uint8
/tmp/ipython-input-1784619546.py:34: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future
plt.title(class_names[int(y_train[i])])
```



```
After preprocessing:
x_train: (50000, 32, 32, 3) float32 min/max: 0.0 1.0
y_train_cat: (50000, 10) float64
```

```
#Build 3 models with 3 different activation functions
# (1 input layer, 1 hidden layer, 1 output layer, hidden has 64 neurons)
```

```
import tensorflow as tf
from keras import backend as K
from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.optimizers import Adam
import pandas as pd
import matplotlib.pyplot as plt

if "x_valid" not in globals() and "x_test" in globals():
    x_valid = x_test
if "y_valid" not in globals() and "y_test" in globals():
    y_valid = y_test

# Ensure labels are one-hot
def ensure_one_hot(y, n_classes=10):
```

```

y = np.array(y)
if y.ndim == 2 and y.shape[1] == n_classes:
    return y
return keras.utils.to_categorical(y.reshape(-1), n_classes)

y_train_oh = ensure_one_hot(y_train, 10)
y_valid_oh = ensure_one_hot(y_valid, 10)

# Pick 3 activations (one MUST be ReLU)
activations_to_try = ["relu", "tanh", "elu"]

activation_reason = {
    "relu": "Fast, simple, and helps reduce vanishing gradients for many problems.",
    "tanh": "Zero-centered outputs can make optimization smoother than sigmoid in some cases.",
    "elu": "Allows small negative outputs which can reduce dead neurons compared to ReLU."
}

def build_mlp(hidden_activation: str, hidden_units: int = 64, lr: float = 1e-3):
    K.clear_session()
    model = Sequential()
    model.add(Flatten(input_shape=(32, 32, 3)))
    model.add(Dense(hidden_units, activation=hidden_activation))
    model.add(Dense(10, activation="softmax"))

    model.compile(
        loss="categorical_crossentropy",
        optimizer=Adam(learning_rate=lr),
        metrics=["accuracy"]
    )
    return model

```

```

# Tune the best model

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from keras.models import Sequential
from keras.layers import Dense, Flatten
from keras.optimizers import Adam, SGD

# Make sure we have train/valid and one hot labels available
if "x_valid" not in globals() and "x_test" in globals():
    x_valid = x_test

if "y_train_oh" not in globals():
    y_train_oh = y_train
if "y_valid_oh" not in globals():
    y_valid_oh = y_valid

if "best_activation" not in globals():
    if "results_df" in globals() and len(results_df) > 0:
        best_activation = results_df.loc[0, "activation"]
    else:
        best_activation = "relu" # fallback

print("Best activation selected for tuning:", best_activation)

def build_mlp_tuned(hidden_activation,
                    hidden_units = 64,
                    add_second_hidden = False,
                    optimizer = None,
                    loss_fn = "categorical_crossentropy",
                    lr = 1e-3):
    """
    1 input layer (Flatten), 1 or 2 hidden Dense layers, 1 softmax output layer.
    """
    model = Sequential()
    model.add(Flatten(input_shape = (32, 32, 3)))
    model.add(Dense(hidden_units, activation = hidden_activation))

    if add_second_hidden:
        model.add(Dense(hidden_units, activation = hidden_activation))

    model.add(Dense(10, activation = "softmax"))

    if optimizer is None:
        optimizer = Adam(learning_rate = lr)

    model.compile(loss = loss_fn, optimizer = optimizer, metrics = ["accuracy"])
    return model

```

```
def fit_and_eval(model, run_name, epochs = 20, batch_size = 128):
    print("\n" + "=" * 70)
    print("Run:", run_name)
    print("=" * 70)

    history = model.fit(x_train, y_train_oh,
                        batch_size = batch_size,
                        epochs = epochs,
                        verbose = 1,
                        validation_data = (x_valid, y_valid_oh))

    val_loss, val_acc = model.evaluate(x_valid, y_valid_oh, verbose = 0)
    print(f"Validation accuracy: {val_acc:.4f}")

    return history, val_loss, val_acc

# Baseline: best activation, 64 units, 1 hidden layer, Adam + categorical crossentropy
baseline_model = build_mlp_tuned(hidden_activation = best_activation,
                                hidden_units = 64,
                                add_second_hidden = False,
                                optimizer = Adam(learning_rate = 1e-3),
                                loss_fn = "categorical_crossentropy")

baseline_hist, baseline_loss, baseline_acc = fit_and_eval(
    baseline_model, run_name = "BASELINE (best activation, 64 units, Adam, crossentropy)"
)

print("\nBaseline validation accuracy:", baseline_acc)
```

Best activation selected for tuning: relu

```
=====
Run: BASELINE (best activation, 64 units, Adam, crossentropy)
=====
Epoch 1/20
391/391 ————— 6s 13ms/step - accuracy: 0.2531 - loss: 2.0702 - val_accuracy: 0.3407 - val_loss: 1.8488
Epoch 2/20
391/391 ————— 4s 9ms/step - accuracy: 0.3492 - loss: 1.8275 - val_accuracy: 0.3606 - val_loss: 1.7968
Epoch 3/20
391/391 ————— 4s 11ms/step - accuracy: 0.3701 - loss: 1.7781 - val_accuracy: 0.3685 - val_loss: 1.7776
Epoch 4/20
391/391 ————— 4s 11ms/step - accuracy: 0.3824 - loss: 1.7465 - val_accuracy: 0.3653 - val_loss: 1.7742
Epoch 5/20
391/391 ————— 4s 9ms/step - accuracy: 0.3903 - loss: 1.7268 - val_accuracy: 0.3729 - val_loss: 1.7571
Epoch 6/20
391/391 ————— 4s 10ms/step - accuracy: 0.3969 - loss: 1.7102 - val_accuracy: 0.3763 - val_loss: 1.7413
Epoch 7/20
391/391 ————— 5s 12ms/step - accuracy: 0.4002 - loss: 1.6952 - val_accuracy: 0.3857 - val_loss: 1.7282
Epoch 8/20
391/391 ————— 4s 9ms/step - accuracy: 0.4040 - loss: 1.6829 - val_accuracy: 0.3914 - val_loss: 1.7085
Epoch 9/20
391/391 ————— 4s 9ms/step - accuracy: 0.4078 - loss: 1.6679 - val_accuracy: 0.3949 - val_loss: 1.6966
Epoch 10/20
391/391 ————— 6s 11ms/step - accuracy: 0.4116 - loss: 1.6538 - val_accuracy: 0.4008 - val_loss: 1.6842
Epoch 11/20
391/391 ————— 4s 9ms/step - accuracy: 0.4167 - loss: 1.6420 - val_accuracy: 0.4048 - val_loss: 1.6718
Epoch 12/20
391/391 ————— 4s 9ms/step - accuracy: 0.4202 - loss: 1.6319 - val_accuracy: 0.4077 - val_loss: 1.6644
Epoch 13/20
391/391 ————— 5s 12ms/step - accuracy: 0.4215 - loss: 1.6246 - val_accuracy: 0.4093 - val_loss: 1.6599
Epoch 14/20
391/391 ————— 4s 9ms/step - accuracy: 0.4240 - loss: 1.6176 - val_accuracy: 0.4102 - val_loss: 1.6588
Epoch 15/20
391/391 ————— 4s 9ms/step - accuracy: 0.4253 - loss: 1.6112 - val_accuracy: 0.4136 - val_loss: 1.6528
Epoch 16/20
391/391 ————— 5s 12ms/step - accuracy: 0.4282 - loss: 1.6063 - val_accuracy: 0.4147 - val_loss: 1.6499
Epoch 17/20
391/391 ————— 4s 9ms/step - accuracy: 0.4287 - loss: 1.6025 - val_accuracy: 0.4140 - val_loss: 1.6514
Epoch 18/20
391/391 ————— 4s 10ms/step - accuracy: 0.4297 - loss: 1.5983 - val_accuracy: 0.4147 - val_loss: 1.6475
Epoch 19/20
391/391 ————— 11s 27ms/step - accuracy: 0.4310 - loss: 1.5938 - val_accuracy: 0.4156 - val_loss: 1.6426
Epoch 20/20
391/391 ————— 8s 22ms/step - accuracy: 0.4322 - loss: 1.5908 - val_accuracy: 0.4157 - val_loss: 1.6435
Validation accuracy: 0.4157
```

Baseline validation accuracy: 0.4156999886035919

```
# Experiments:
# 1) 3 different hidden-unit sizes (ex: 128, 256, 512)
# 2) add 1 additional hidden layer
# 3) 1 different optimizer
# 4) 1 different loss function
```

```

EPOCHS = 20
BATCH_SIZE = 128

tune_results = []
tune_histories = {}

# (1) Hidden units sweep
for units in [128, 256, 512]:
    model = build_mlp_tuned(hidden_activation = best_activation,
                            hidden_units = units,
                            add_second_hidden = False,
                            optimizer = Adam(learning_rate = 1e-3),
                            loss_fn = "categorical_crossentropy")
    hist, loss, acc = fit_and_eval(model, run_name = f"Hidden units = {units}", epochs = EPOCHS, batch_size = BATCH_SIZE)

    tune_histories[f"units_{units}"] = hist
    tune_results.append({
        "experiment": f"hidden_units_{units}",
        "activation": best_activation,
        "hidden_units": units,
        "extra_hidden_layer": False,
        "optimizer": "Adam(lr=1e-3)",
        "loss": "categorical_crossentropy",
        "val_accuracy": acc,
        "val_loss": loss
    })

# (2) Add 1 additional hidden layer (keep 64 units to isolate effect)
model = build_mlp_tuned(hidden_activation = best_activation,
                        hidden_units = 64,
                        add_second_hidden = True,
                        optimizer = Adam(learning_rate = 1e-3),
                        loss_fn = "categorical_crossentropy")
hist, loss, acc = fit_and_eval(model, run_name = "Add 1 extra hidden layer (64 units + 64 units)", epochs = EPOCHS, batch_size = BATCH_SIZE)

tune_histories["extra_layer"] = hist
tune_results.append({
    "experiment": "extra_hidden_layer",
    "activation": best_activation,
    "hidden_units": 64,
    "extra_hidden_layer": True,
    "optimizer": "Adam(lr=1e-3)",
    "loss": "categorical_crossentropy",
    "val_accuracy": acc,
    "val_loss": loss
})

# (3) Different optimizer (use SGD with momentum)
sgd_opt = SGD(learning_rate = 1e-2, momentum = 0.9)
model = build_mlp_tuned(hidden_activation = best_activation,
                        hidden_units = 64,
                        add_second_hidden = False,
                        optimizer = sgd_opt,
                        loss_fn = "categorical_crossentropy")
hist, loss, acc = fit_and_eval(model, run_name = "Different optimizer: SGD(lr=1e-2, momentum=0.9)", epochs = EPOCHS, batch_size = BATCH_SIZE)

tune_histories["sgd_optimizer"] = hist
tune_results.append({
    "experiment": "different_optimizer_SGD",
    "activation": best_activation,
    "hidden_units": 64,
    "extra_hidden_layer": False,
    "optimizer": "SGD(lr=1e-2,m=0.9)",
    "loss": "categorical_crossentropy",
    "val_accuracy": acc,
    "val_loss": loss
})

# (4) Different loss function (use MSE / quadratic cost style)
model = build_mlp_tuned(hidden_activation = best_activation,
                        hidden_units = 64,
                        add_second_hidden = False,
                        optimizer = Adam(learning_rate = 1e-3),
                        loss_fn = "mean_squared_error")
hist, loss, acc = fit_and_eval(model, run_name = "Different loss: mean_squared_error", epochs = EPOCHS, batch_size = BATCH_SIZE)

tune_histories["mse_loss"] = hist
tune_results.append({
    "experiment": "different_loss_MSE",
    "activation": best_activation,
    "hidden_units": 64,
    "extra_hidden_layer": False,
    "optimizer": "Adam(lr=1e-3)",

```

```

"loss": "mean_squared_error",
"val_accuracy": acc,
"val_loss": loss
})

# Summary table
tune_df = pd.DataFrame(tune_results).sort_values(by = "val_accuracy", ascending = False).reset_index(drop = True)

# Add improvement columns vs baseline
tune_df["baseline_val_accuracy"] = baseline_acc
tune_df["delta_accuracy"] = tune_df["val_accuracy"] - baseline_acc
tune_df["improved_vs_baseline"] = tune_df["delta_accuracy"] > 0

print("\nTuning results (sorted by val_accuracy):")
display(tune_df)

best_tuned = tune_df.loc[0]
print("\nBest tuned setup:")
print(best_tuned)

# quick plot of validation accuracy for each tuning run
plt.figure(figsize = (8, 4))
plt.bar(tune_df["experiment"], tune_df["val_accuracy"])
plt.xticks(rotation = 45, ha = "right")
plt.ylabel("Validation Accuracy")
plt.title("Step 5 Tuning Results (Validation Accuracy)")
plt.tight_layout()
plt.show()

```

```

=====
Run: Hidden units = 128
=====
Epoch 1/20
391/391 ————— 8s 17ms/step - accuracy: 0.2665 - loss: 2.0613 - val_accuracy: 0.3595 - val_loss: 1.7966
Epoch 2/20
391/391 ————— 5s 12ms/step - accuracy: 0.3659 - loss: 1.7808 - val_accuracy: 0.3874 - val_loss: 1.7190
Epoch 3/20
391/391 ————— 6s 16ms/step - accuracy: 0.3952 - loss: 1.7065 - val_accuracy: 0.4007 - val_loss: 1.6777
Epoch 4/20
391/391 ————— 9s 13ms/step - accuracy: 0.4113 - loss: 1.6606 - val_accuracy: 0.4071 - val_loss: 1.6514
Epoch 5/20
391/391 ————— 10s 13ms/step - accuracy: 0.4228 - loss: 1.6271 - val_accuracy: 0.4135 - val_loss: 1.6329
Epoch 6/20
391/391 ————— 6s 16ms/step - accuracy: 0.4293 - loss: 1.6034 - val_accuracy: 0.4210 - val_loss: 1.6162
Epoch 7/20
391/391 ————— 5s 13ms/step - accuracy: 0.4387 - loss: 1.5818 - val_accuracy: 0.4290 - val_loss: 1.5885
Epoch 8/20
391/391 ————— 6s 16ms/step - accuracy: 0.4483 - loss: 1.5620 - val_accuracy: 0.4316 - val_loss: 1.5892
Epoch 9/20
391/391 ————— 9s 13ms/step - accuracy: 0.4504 - loss: 1.5466 - val_accuracy: 0.4340 - val_loss: 1.5818
Epoch 10/20
391/391 ————— 6s 16ms/step - accuracy: 0.4574 - loss: 1.5320 - val_accuracy: 0.4435 - val_loss: 1.5666
Epoch 11/20
391/391 ————— 5s 13ms/step - accuracy: 0.4613 - loss: 1.5205 - val_accuracy: 0.4450 - val_loss: 1.5646
Epoch 12/20
391/391 ————— 6s 16ms/step - accuracy: 0.4636 - loss: 1.5088 - val_accuracy: 0.4491 - val_loss: 1.5567
Epoch 13/20
391/391 ————— 6s 15ms/step - accuracy: 0.4678 - loss: 1.4990 - val_accuracy: 0.4478 - val_loss: 1.5616
Epoch 14/20
391/391 ————— 6s 15ms/step - accuracy: 0.4702 - loss: 1.4895 - val_accuracy: 0.4498 - val_loss: 1.5510
Epoch 15/20
391/391 ————— 5s 14ms/step - accuracy: 0.4756 - loss: 1.4814 - val_accuracy: 0.4503 - val_loss: 1.5494
Epoch 16/20
391/391 ————— 5s 13ms/step - accuracy: 0.4781 - loss: 1.4745 - val_accuracy: 0.4530 - val_loss: 1.5459
Epoch 17/20
391/391 ————— 6s 15ms/step - accuracy: 0.4824 - loss: 1.4663 - val_accuracy: 0.4560 - val_loss: 1.5401
Epoch 18/20
391/391 ————— 5s 13ms/step - accuracy: 0.4848 - loss: 1.4593 - val_accuracy: 0.4575 - val_loss: 1.5390
Epoch 19/20
391/391 ————— 6s 16ms/step - accuracy: 0.4863 - loss: 1.4538 - val_accuracy: 0.4571 - val_loss: 1.5374
Epoch 20/20
391/391 ————— 5s 13ms/step - accuracy: 0.4877 - loss: 1.4488 - val_accuracy: 0.4612 - val_loss: 1.5369
Validation accuracy: 0.4612

```

```

=====
Run: Hidden units = 256
=====
Epoch 1/20
391/391 ————— 11s 25ms/step - accuracy: 0.2730 - loss: 2.0793 - val_accuracy: 0.3594 - val_loss: 1.7850
Epoch 2/20
391/391 ————— 9s 24ms/step - accuracy: 0.3739 - loss: 1.7661 - val_accuracy: 0.3898 - val_loss: 1.7025
Epoch 3/20
391/391 ————— 9s 23ms/step - accuracy: 0.3995 - loss: 1.6968 - val_accuracy: 0.4045 - val_loss: 1.6526
Epoch 4/20
391/391 ————— 9s 21ms/step - accuracy: 0.4165 - loss: 1.6440 - val_accuracy: 0.4272 - val_loss: 1.6080
Epoch 5/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 6/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 7/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 8/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 9/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 10/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 11/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 12/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 13/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 14/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 15/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 16/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 17/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 18/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 19/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Epoch 20/20
391/391 ————— 9s 22ms/step - accuracy: 0.4211 - loss: 1.6040 - val_accuracy: 0.4227 - val_loss: 1.5880
Validation accuracy: 0.4227

```

```

391/391 9s 23ms/step - accuracy: 0.4391 - loss: 1.5722 - val_accuracy: 0.4436 - val_loss: 1.5550
The baseline training model used ReLU, one hidden layer with 64 neurons, Adam with learning rate 0.001, and categorical crossentropy. Its validation accuracy was about 0.4116. When we increased the hidden layer size, validation accuracy improved each time. With 128 hidden units the validation accuracy reached 0.4546. With 256 hidden units it reached 0.4827. With 512 hidden units it reached 0.4835 which was the best value shown in the results. The jump from 64 to 256 gave most of the benefit, and the improvement from 256 to 512 was very small. For the simple MLP started to hit diminishing returns. The added hidden layer run with 64 and 64 units was improving during the visible epochs, reaching a validation accuracy of about 0.4532 by epoch 12, which is above the baseline. The provided output capture did not include the final validation accuracy line for that run, and it did not show the printed results for the different optimizers run on the different loss function run. So we can only confirm the accuracy changes for the baseline and the hidden unit sweep.
Epoch 8/20
391/391 9s 24ms/step - accuracy: 0.4603 - loss: 1.5217 - val_accuracy: 0.4477 - val_loss: 1.5371
Epoch 9/20
391/391 9s 23ms/step - accuracy: 0.4658 - loss: 1.5007 - val_accuracy: 0.4461 - val_loss: 1.5280
Epoch 10/20
391/391 9s 24ms/step - accuracy: 0.4823 - loss: 1.4614 - val_accuracy: 0.4597 - val_loss: 1.4994
Epoch 11/20
391/391 12s 29ms/step - accuracy: 0.4860 - loss: 1.4499 - val_accuracy: 0.4632 - val_loss: 1.4943
Epoch 12/20
391/391 9s 23ms/step - accuracy: 0.4902 - loss: 1.4361 - val_accuracy: 0.4722 - val_loss: 1.4819
Epoch 13/20
391/391 8s 21ms/step - accuracy: 0.4955 - loss: 1.4232 - val_accuracy: 0.4664 - val_loss: 1.4907
Epoch 14/20
391/391 9s 24ms/step - accuracy: 0.4977 - loss: 1.4145 - val_accuracy: 0.4753 - val_loss: 1.4749
Epoch 15/20
391/391 9s 24ms/step - accuracy: 0.5017 - loss: 1.4038 - val_accuracy: 0.4773 - val_loss: 1.4721
Epoch 16/20
391/391 8s 21ms/step - accuracy: 0.5068 - loss: 1.3912 - val_accuracy: 0.4732 - val_loss: 1.4771
Epoch 17/20
391/391 9s 24ms/step - accuracy: 0.5076 - loss: 1.3841 - val_accuracy: 0.4733 - val_loss: 1.4818
Epoch 18/20
391/391 11s 25ms/step - accuracy: 0.5097 - loss: 1.3759 - val_accuracy: 0.4763 - val_loss: 1.4744
Epoch 19/20
391/391 8s 21ms/step - accuracy: 0.5138 - loss: 1.3678 - val_accuracy: 0.4751 - val_loss: 1.4837
Validation accuracy: 0.4751

```

```

=====
Run: Hidden units = 512

```